데이터베이스(SQL)

11장. DDL

학습 목표

1) DDL의 역할과 다루는 객체(테이블·뷰·인덱스·스키마)를 설명할 수 있다.
2) CREATE TABLE로 테이블을 만들 수 있다.
3) PRIMARY KEY, NOT NULL, UNIQUE, CHECK, DEFAULT, FOREIGN KEY의 역할을 구분할 수 있다.
4) ALTER TABLE로 컬럼과 제약조건을 변경할 수 있다.
5) TRUNCATE와 DROP의 차이를 설명할 수 있다.
6) PostgreSQL에서 DDL도 트랜잭션 안에서 롤백 가능한 경우가 있다는 것을 확인할 수 있다.

전체 흐름

1) DDL이란 → 2) 데이터 무결성과 제약조건 → 3) 기본 예제와 제조 예제 → 4) ALTER TABLE

5) TRUNCATE와 DROP → 6) DDL과 트랜잭션

01 DDL이란

DDL

DDL은 테이블·뷰·인덱스·스키마 같은 데이터베이스 객체를 만들고(CREATE), 고치고(ALTER), 지우는(DROP) SQL이다.

명령	역할
CREATE	테이블 등 객체 생성
ALTER	기존 객체 구조 변경
TRUNCATE	테이블 구조는 유지하고 모든 행 제거
DROP	객체 자체 삭제

공식 문서: 데이터 정의 / 테이블 기초 / CREATE TABLE

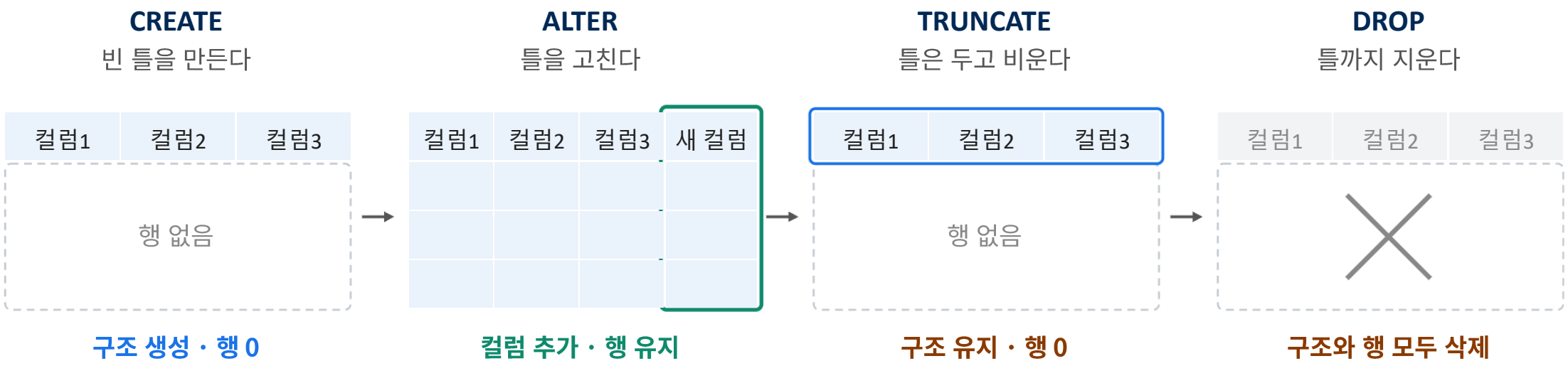
DDL이 다루는 객체

객체	무엇인가
스키마	객체를 묶는 폴더 (public, mfg, pdm)
테이블	데이터를 담는 틀
뷰	저장해 둔 조회 쿼리
인덱스	검색을 빠르게 하는 보조 자료
시퀀스	번호를 생성하는 객체 (IDENTITY 열에서 사용)

형식: CREATE 종류 이름 … / ALTER 종류 이름 … / DROP 종류 이름

공식 문서: 데이터 정의

DDL 명령어 — 테이블 기준



CREATE TABLE 기본 형식

```
CREATE TABLE [IF NOT EXISTS] 테이블명 (
컬럼명1 데이터타입 [컬럼 제약조건],
컬럼명2 데이터타입 [컬럼 제약조건],
[테이블 제약조건]
);
```

부분	뜻
테이블명	만들 테이블 이름
컬럼명 데이터타입	컬럼 이름과 담을 값의 종류 (INTEGER, VARCHAR(50), NUMERIC(10,2), DATE 등)
컬럼 제약조건	컬럼 정의 바로 뒤에 쓰는 제약조건
테이블 제약조건	컬럼 목록 뒤에 따로 쓰는 제약조건 (여러 컬럼에 걸칠 때)
IF NOT EXISTS	같은 이름의 테이블이 이미 있으면 오류 대신 알림만 내고 넘어간다

대괄호 [ ]는 생략할 수 있는 부분이다. 컬럼 정의 사이는 쉼표로 구분하고, 마지막 항목 뒤에는 쉼표를 쓰지 않는다.

02 데이터 무결성과 제약조건

제약조건이 필요한 이유

잘못된 데이터가 들어오면 분석 결과도 잘못된다. 제약조건은 테이블에 들어오는 값을 검사해 **규칙에 맞지 않는 값을 입력 단계에서 거부**한다.

제약조건	의미
PRIMARY KEY	행을 고유하게 식별
NOT NULL	NULL 입력 방지
UNIQUE	중복 값 방지
CHECK	업무 규칙 범위 검사
DEFAULT	값 생략 시 기본값
FOREIGN KEY	다른 테이블의 유효한 키 참조

예: score INTEGER CHECK (score BETWEEN 0 AND 100) — 점수는 0~100만 받는다.

공식 문서: 제약조건

제약조건과 DEFAULT 작성 방식

컬럼 제약조건·DEFAULT — 컬럼 정의 뒤

컬럼명 데이터타입 PRIMARY KEY
컬럼명 데이터타입 NOT NULL
컬럼명 데이터타입 UNIQUE
컬럼명 데이터타입 CHECK (조건식)
컬럼명 데이터타입 DEFAULT 기본값
컬럼명 데이터타입 REFERENCES 부모테이블 (부모컬럼)

테이블 제약조건 — 컬럼 목록 뒤에 따로

PRIMARY KEY (컬럼1, 컬럼2)
UNIQUE (컬럼1, 컬럼2)
CHECK (조건식)
FOREIGN KEY (컬럼명)
REFERENCES 부모테이블 (부모컬럼)

제약조건에 이름을 붙일 때: CONSTRAINT 제약조건명 CHECK (조건식). 이름을 붙이지 않으면 PostgreSQL이 자동으로 이름을 정한다.
NOT NULL은 컬럼 제약조건이고, DEFAULT는 기본값 설정이다. 둘 다 컬럼 정의에 작성한다.

PRIMARY KEY

- 행의 **대표 식별자**다.

- **중복 불가 + NULL 불가**. UNIQUE와 NOT NULL을 함께 건 것과 같은 규칙이다.

- 테이블당 PRIMARY KEY 제약조건은 **하나**만 둘 수 있다.

- 여러 컬럼을 묶은 **복합 PK**는 가능하다.

- PK를 만들면 고유성을 보장하기 위한 인덱스가 자동으로 생긴다.

- PK로 자주 쓰는 자동 증가 번호(IDENTITY 열) -행을 넣을 때 번호를 자동으로 붙이므로 INSERT에서 값을 생략한다.

단일 PK

```
컬럼명 데이터타입 PRIMARY KEY
```

복합 PK

```
CREATE TABLE 테이블명 (
        컬럼1 데이터타입,
        컬럼2 데이터타입,
        PRIMARY KEY (컬럼1, 컬럼2)
);
```

자동 증가 번호 PK

```
id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY
```

NOT NULL · UNIQUE · DEFAULT

구분	형식	설명
NOT NULL	컬럼명 데이터타입 NOT NULL	NULL 허용 안 함
UNIQUE	컬럼명 데이터타입 UNIQUE UNIQUE (컬럼1, 컬럼2)	같은 값(또는 같은 조합) 중복 허용 안 함
DEFAULT	컬럼명 데이터타입 DEFAULT 기본값	기본값 적용

- UNIQUE도 PK처럼 고유성을 보장하기 위한 인덱스가 자동으로 생긴다.

- DEFAULT가 없으면 생략된 값은 NULL로 처리된다 (NOT NULL이면 오류).

- 기본값 예: DEFAULT 0, DEFAULT 'Y', DEFAULT CURRENT_DATE

CHECK와 NOT NULL

- **CHECK는 업무 규칙**이다. 괄호 안 조건식으로 값의 범위나 관계를 검사한다.

- **NULL 금지는 NOT NULL**로 따로 건다.

```
컬럼명 데이터타입 CHECK (조건식)

score INTEGER CHECK (score BETWEEN 0 AND 100)
score INTEGER NOT NULL CHECK (score BETWEEN 0 AND 100)
```

FOREIGN KEY

- **부모/자식 관계**: 자식 테이블의 컬럼이 부모 테이블의 키를 가리킨다.

- **참조 무결성**: 부모에 없는 키 값은 자식에 들어갈 수 없다.

- 부모 쪽 참조 컬럼은 PRIMARY KEY나 UNIQUE여야 한다.

```
-- 컬럼 뒤에 쓰는 형식
컬럼명 데이터타입 REFERENCES 부모테이블 (부모컬럼)

-- 테이블 제약조건 형식
FOREIGN KEY (컬럼명) REFERENCES 부모테이블 (부모컬럼)
[ON DELETE 동작] [ON UPDATE 동작]
```

참조 동작 — ON DELETE

동작	부모 행을 삭제할 때
NO ACTION (지정하지 않으면 이 동작)	참조하는 자식 행이 남아 있으면 오류를 내고 삭제를 거부한다
RESTRICT	NO ACTION과 같이 삭제를 거부한다. 다만 즉시 검사한다
CASCADE	참조하는 자식 행을 함께 삭제한다
SET NULL	자식 행의 FK 컬럼을 NULL로 바꾼다
SET DEFAULT	자식 행의 FK 컬럼을 그 컬럼의 DEFAULT 값으로 바꾼다

부모 키 값을 바꿀 때의 동작은 ON UPDATE로 같은 방식으로 정한다. CASCADE는 자식 데이터가 한꺼번에 사라질 수 있으므로 신중하게 쓴다
.

제약조건과 자동 인덱스

제약조건	인덱스 자동 생성	의미
PRIMARY KEY	생성됨	고유성을 보장하기 위한 인덱스가 함께 만들어진다
UNIQUE	생성됨	고유성을 보장하기 위한 인덱스가 함께 만들어진다
FOREIGN KEY (참조하는 쪽)	생성되지 않음	자식 테이블의 FK 컬럼에는 인덱스가 자동으로 생기지 않는다

03 기본 예제와 제조 예제

문법은 같고 의미만 다르다

기본 예제에서는 edu_customer, 제조 예제에서는 edu_machine, edu_machine_event를 만든다. 문법은 같고 **데이터 의미만 달라진다.**

	기본 예제	제조 예제
테이블	edu_customer	edu_machine, edu_machine_event
다루는 대상	고객	설비 마스터와 설비 이벤트
쓰는 문법	CREATE TABLE, 제약조건	CREATE TABLE, 제약조건, FOREIGN KEY
달라지는 것	컬럼의 업무 의미	컬럼의 업무 의미

공식 문서: 제약조건

edu_machine → edu_machine_event



참조 무결성
부모에 없는 키는 자식에 들어갈 수 없다

주의
자식 FK 컬럼의 인덱스는 자동 생성되지 않는다

04 ALTER TABLE

ALTER TABLE

하는 일	형식
컬럼 추가	ALTER TABLE 테이블명 ADD COLUMN 컬럼명 데이터타입 [제약조건];
기본값 지정	ALTER TABLE 테이블명 ALTER COLUMN 컬럼명 SET DEFAULT 기본값;
컬럼 이름 변경	ALTER TABLE 테이블명 RENAME COLUMN 기존컬럼명 TO 새컬럼명;

- ADD COLUMN으로 추가한 컬럼은 기존 행에서 기본값(없으면 NULL)으로 채워진다.

- SET DEFAULT는 **이후에 넣는 행**부터 적용된다. 이미 들어 있는 행의 값은 바뀌지 않는다.

- RENAME은 이름만 바꾸고 저장된 데이터에는 영향이 없다.

공식 문서: ALTER TABLE / 테이블 수정

ALTER TABLE 형식 - 컬럼과 테이블

하는 일	형식
컬럼 추가	ALTER TABLE 테이블명 ADD COLUMN 컬럼명 데이터타입;
컬럼 삭제	ALTER TABLE 테이블명 DROP COLUMN 컬럼명;
데이터 타입 변경	ALTER TABLE 테이블명 ALTER COLUMN 컬럼명 TYPE 새데이터타입;
컬럼 이름 변경	ALTER TABLE 테이블명 RENAME COLUMN 기존컬럼명 TO 새컬럼명;
테이블 이름 변경	ALTER TABLE 테이블명 RENAME TO 새테이블명;

옵션: ADD COLUMN IF NOT EXISTS, DROP COLUMN IF EXISTS는 컬럼이 이미 있거나 없을 때 오류 대신 알림만 낸다. 타입을 바꿀 때 기존 값을 새 타입으로 바꿀 수 없으면 오류가 난다.

ALTER TABLE 형식 - 기본값과 제약조건

하는 일	형식
기본값 지정 / 제거	ALTER TABLE 테이블명 ALTER COLUMN 컬럼명 SET DEFAULT 기본값; ALTER TABLE 테이블명 ALTER COLUMN 컬럼명 DROP DEFAULT;
NOT NULL 지정 / 해제	ALTER TABLE 테이블명 ALTER COLUMN 컬럼명 SET NOT NULL; ALTER TABLE 테이블명 ALTER COLUMN 컬럼명 DROP NOT NULL;
제약조건 추가	ALTER TABLE 테이블명 ADD CONSTRAINT 제약조건명 CHECK (조건식);
제약조건 삭제	ALTER TABLE 테이블명 DROP CONSTRAINT 제약조건명;

- SET NOT NULL은 그 컬럼에 이미 NULL이 있으면 실패한다.

- ADD CONSTRAINT는 기존 행이 규칙을 어기면 실패한다. 제약조건 자리에 UNIQUE (컬럼), FOREIGN KEY (컬럼) REFERENCES …도 쓸 수 있다.

- 제약조건을 지우려면 이름이 필요하다. 그래서 만들 때 이름을 붙여 두면 편하다.

05 TRUNCATE와 DROP

TRUNCATE vs DROP

구분	TRUNCATE	DROP
데이터	전체 제거	객체와 함께 제거
구조	유지	삭제
다시 쓰려면	바로 INSERT 가능	CREATE TABLE부터 다시
기본 형식	TRUNCATE TABLE 테이블명;	DROP TABLE 테이블명;

중간 결과 테이블을 다시 채울 때는 TRUNCATE, 더 이상 쓰지 않는 테이블은 DROP이다.

공식 문서: TRUNCATE / DROP TABLE

TRUNCATE 형식과 옵션

```
TRUNCATE TABLE 테이블명;
TRUNCATE TABLE 테이블명 RESTART IDENTITY;
TRUNCATE TABLE 부모테이블 CASCADE;
```

옵션	뜻
CONTINUE IDENTITY (기본값)	자동 증가 번호(IDENTITY 열)를 그대로 이어 간다
RESTART IDENTITY	컬럼에 소유된 시퀀스를 설정된 시작값으로 초기화한다
RESTRICT (기본값)	다른 테이블이 FK로 참조하고 있으면 거부한다
CASCADE	FK로 참조하는 자식 테이블도 함께 비운다

TRUNCATE도 트랜잭션 안에서 실행하면 커밋 전에 롤백으로 되돌릴 수 있다.

DROP TABLE 형식과 옵션

```
DROP TABLE 테이블명;
DROP TABLE IF EXISTS 테이블명;
DROP TABLE 테이블명 CASCADE;
```

옵션	뜻
IF EXISTS	테이블이 없어도 오류 대신 알림만 내고 넘어간다
RESTRICT (기본값)	다른 객체(뷰, 다른 테이블의 FK)가 의존하고 있으면 거부한다
CASCADE	의존하는 객체도 함께 지운다. 다른 테이블의 FK는 제약조건만 지우고 그 테이블은 남긴다

테이블을 지우면 그 테이블의 인덱스와 제약조건도 함께 지워진다.

06 DDL과 트랜잭션

DDL 트랜잭션 롤백

```
BEGIN;

CREATE TABLE 테이블명 (
컬럼명 데이터타입
);

ROLLBACK;
```

- PostgreSQL에서는 **많은 DDL이 트랜잭션에서 롤백 가능**하다.
- BEGIN으로 트랜잭션을 시작하고 ROLLBACK하면 만든 테이블이 남지 않는다.
- COMMIT하면 그때 확정된다.

	Windows	macOS
세 문장을 한 번에 실행 (스크립트 실행)	Alt + X	Option + X

공식 문서: 트랜잭션

DDL 트랜잭션 롤백

많은 DDL은 트랜잭션에서 롤백할 수 있지만, **트랜잭션 블록(BEGIN ~ COMMIT) 안에서 실행할 수 없는 명령**도 있다.

명령	트랜잭션 블록 안에서
CREATE TABLE, ALTER TABLE, DROP TABLE, TRUNCATE	실행 가능 (롤백 가능)
CREATE INDEX	실행 가능
CREATE DATABASE	실행할 수 없음

블록 안에서 실행하면 나오는 오류

```
ERROR: CREATE DATABASE cannot be run inside a transaction block
```

핵심 정리

1) DDL = 구조. 테이블·뷰·인덱스·스키마 같은 객체를 만들고 고치고 지운다.

2) PK는 테이블당 하나의 PRIMARY KEY 제약조건만 둘 수 있지만 복합 PK는 가능하다.

3) PK/UNIQUE는 PostgreSQL에서 고유성을 보장하기 위한 인덱스를 자동 생성한다.

4) FK를 만든다고 **참조하는 쪽 FK 컬럼의 인덱스가 자동 생성되는 것은 아니다.**

5) PostgreSQL의 많은 DDL은 트랜잭션에서 롤백할 수 있지만 모든 명령이 트랜잭션 블록에서 허용되는 것은 아니다.